

vllm专题（一）：安装-GPU

原创 无声之钟 已于 2025-03-

分类专栏：大模型专题系列



大模型专题系列 专栏收

您已是超级会员，正在免费阅读该会员专享内容

查看更多超级会员权益 >

vLLM是高性能LLM推理和部署库。
支持NVIDIA CUDA, AMD ROCm, 和Intel XPU三大平台GPU
NVIDIA最方便, AMD和Intel目前更多手动操作
(理解背后的原理和注意事项, 更好的部署使用vLLM)
安装方式:
预构建Wheel包 (NVIDIA)
Docker镜像 (NVIDIA/AMD)
源码构建 (所有平台)

¥49.90
¥99.00

97 篇文章

已订阅

8折续

vLLM 是一个 Python 库，支持以下 GPU 变体。选择您的 GPU 类型以查看供应商特定的说明：

1. NVIDIA CUDA

vLLM 包含预编译的 C++ 和 CUDA（12.1）二进制文件。

2. AMD ROCm

vLLM 支持配备 ROCm 6.3 的 AMD GPU。

注意

此设备没有预构建的 wheel 包，因此您必须使用预构建的 Docker 镜像或从源代码构建 vLLM。

3. Intel XPU

vLLM 初步支持在 Intel GPU 平台上进行基本模型推理和服务。

注意

此设备没有预构建的 wheel 包或镜像，因此您必须从源代码构建 vLLM。

NVIDIA提供wheel包，能直接下载
pip install vllm
AMD没有构建wheel包，需要从
docker或源码中构建
Intel也是类似如此

1. 要求

- 操作系统：Linux
- Python：3.9 – 3.12

1. NVIDIA CUDA

- GPU：计算能力 7.0 或更高（例如 V100、T4、RTX20xx、A100、L4、H100 等）

2. AMD ROCm

- GPU：MI200s (gfx90a)、MI300 (gfx942)、Radeon RX 7900 系列 (gfx1100)

- ROCm 6.3

3. Intel XPU

- 支持的硬件：Intel 数据中心 GPU、Intel ARC GPU
- OneAPI 要求：oneAPI 2024.2

创建环境conda或者uv
注意conda仅用于创建环境，不推荐安装包
目前不推荐使用conda来安装包（pytorch
弃用conda渠道）

2. 使用 Python 进行设置

2.1 创建一个新的 Python 环境

您可以使用 `conda` 创建一个新的 Python 环境：

```
# (Recommended) Create a new conda environment.  
conda create -n myenv python=3.12 -y  
conda activate myenv
```

注意

PyTorch 已弃用 conda 发布渠道。如果您使用 conda，请仅用于创建 Python 环境，而不要用于安装包。

或者，您可以使用 `uv`（一个非常快速的 Python 环境管理器）来创建新的 Python 环境。请按照文档安装 `uv`。安装完成后，您可以使用以下命令创建新的 Python 环境：

```
# (Recommended) Create a new uv environment. Use `--seed` to install `pip` and `setuptools` in the environment.  
uv venv myenv --python 3.12 --seed  
source myenv/bin/activate
```

（1）NVIDIA CUDA

注意

通过 `conda` 安装的 PyTorch 会静态链接 NCCL 库，这可能会导致 vLLM 尝试使用 NCCL 时出现问题。更多详情请参阅 [Issue #8420](#)。

为了获得最佳性能，vLLM 需要编译许多 CUDA 内核。不幸的是，这种编译会引入与其他 CUDA 版本和 PyTorch 版本的二进制不兼容性，即使是相同 PyTorch 版本但构建配置不同也会出现问题。

因此，建议在一个全新的环境中安装 vLLM。如果您使用的是不同的 CUDA 版本，或者希望使用现有的 PyTorch 安装，则需要从源代码构建 vLLM。更多详情请参阅下文。

（2）AMD ROCm

此设备没有关于创建新 Python 环境的额外信息。

（3）Intel XPU

此设备没有关于创建新 Python 环境的额外信息。

2.2 预构建的 wheel 包

（1）NVIDIA CUDA

您可以使用 `pip` 或 `uv pip` 安装 vLLM：

vllm 团队将 C++ 和 CUDA 代码编译好了，打包成 wheel 文件。只需要 pip 下载安装就行

```
# Install vLLM with CUDA 12.1.
pip install vllm # If you are using pip.
uv pip install vllm # If you are using uv.
```

截至目前，vLLM 的二进制文件默认使用 CUDA 12.1 和公共 PyTorch 发布版本编译。我们还提供了使用 CUDA 11.8 和公共 PyTorch 发布版本编译的 vLLM 二进制文件：

```
# Install vLLM with CUDA 11.8.
export VLLM_VERSION=0.6.1.post1
export PYTHON_VERSION=310
pip install https://github.com/vllm-project/vllm/releases/download/v${VLLM_VERSION}/vllm-
${VLLM_VERSION}+cu118-cp${PYTHON_VERSION}-cp${PYTHON_VERSION}-manylinux1_x86_64.whl --ext
ra-index-url https://download.pytorch.org/whl/cu118
```

(2) AMD ROCm

目前没有预构建的 ROCm wheel 包。

(3) Intel XPU

目前没有预构建的 XPU wheel 包。

手动设置环境变量，
用 pip install 命令去
指定一个 wheel 文件
url

2.3 从源代码构建 wheel 包

(1) NVIDIA CUDA

使用纯 Python 构建（无需编译）进行设置

如果您只需要修改 Python 代码，可以在不编译的情况下构建和安装 vLLM。使用 pip 的 `--editable` 标志，您对代码所做的更改将在运行 vLLM 时生效：

```
git clone https://github.com/vllm-project/vllm.git
cd vllm
VLLM_USE_PRECOMPILED=1 pip install --editable .
```

此命令将执行以下操作：

1. 在您的 vLLM 克隆中查找当前分支。
2. 识别主分支中对应的基础提交（commit）。
3. 下载基础提交的预构建 wheel 包。
4. 在安装中使用其编译的库。

注意

如果您修改了 C++ 或内核代码，则不能使用纯 Python 构建；否则您将看到关于库未找到或未定义符号的导入错误。

如果您在运行上述命令时看到关于 [wheel 包未找到的错误](#)，可能是因为您基于的主分支提交刚刚合并，[wheel 包正在构建中](#)。在这种情况下，您可以等待大约一小时再重试，或者使用 [VLLM_PRECOMPILED_WHEEL_LOCATION](#) 环境变量手动指定前一个提交进行安装。

```
export VLLM_COMMIT=72d9c316d3f6ede485146fe5aabd4e61dbc59069 # use full commit hash from the main branch
export VLLM_PRECOMPILED_WHEEL_LOCATION=https://wheels.vllm.ai/${VLLM_COMMIT}/vllm-1.0.0.dev-cp38-abi3-manylinux1_x86_64.whl
pip install --editable .
```

您可以在 [安装最新代码](#) 中找到有关 vLLM wheel 包的更多信息。

注意

您的源代码可能与最新的 vLLM wheel 包具有不同的提交 ID，这可能会导致未知错误。建议使用与已安装的 vLLM wheel 包相同的提交 ID 作为源代码。[有关如何安装指定 wheel 包的说明，请参阅 \[安装最新代码\]\(#\)](#)。

完整构建（包括编译）

如果你想修改 C++ 或 CUDA 代码，你需要[从源代码构建 vLLM](#)。这可能需要几分钟时间。

```
git clone https://github.com/vllm-project/vllm.git
cd vllm
pip install -e .
```

提示

从源代码构建需要大量的编译。如果你[反复从源代码构建](#)，使用缓存编译结果会更高效。例如，你可以使用 [conda install ccache](#) 或 [apt install ccache](#) 安装 ccache。只要 [which ccache](#) 命令能找到 ccache 二进制文件，构建系统就会自动使用它。在第一次构建之后，后续的构建会快得多。

[sccache](#) 的工作原理类似于 ccache，但它能够利用远程存储环境中的缓存。可以设置以下环境变量来配置 vLLM 的 sccache 远程存储：

```
SCCACHE_BUCKET=vllm-build-sccache
```

```
SCCACHE_REGION=us-west-2
```

```
SCCACHE_S3_NO_CREDENTIALS=1
```

我们还建议设置 [SCCACHE_IDLE_TIMEOUT=0](#)。

使用现有的 PyTorch 安装

在某些情况下，PyTorch 依赖项无法通过 pip 轻松安装，例如：

- 使用 PyTorch nightly 或自定义 PyTorch 构建来构建 vLLM。
- 使用 aarch64 和 CUDA（GH200）来构建 vLLM，此时 PyTorch 的 wheel 文件在 PyPI 上不可用。目前，只有 PyTorch nightly 为 aarch64 和 CUDA 提供了 wheel 文件。你可以运行 `pip3 install --pre torch torchvision torchaudio --index-url https://download.pytorch.org/whl/nightly/cu124` 来安装 PyTorch nightly，然后在此基础上构建 vLLM。

要使用现有的 PyTorch 安装来构建 vLLM：

```
git clone https://github.com/vllm-project/vllm.git
cd vllm
python use_existing_torch.py
pip install -r requirements-build.txt
pip install -e . --no-build-isolation
```

使用 本地 cutlass 进行编译

目前，在开始构建过程之前，vLLM 会从 GitHub 获取 cutlass 代码。然而，在某些情况下，你可能希望使用本地版本的 cutlass。为此，你可以设置环境变量 VLLM_CUTLASS_SRC_DIR，指向你的本地 cutlass 目录。

```
git clone https://github.com/vllm-project/vllm.git
cd vllm
VLLM_CUTLASS_SRC_DIR=/path/to/cutlass pip install -e .
```

故障排除

编译任务太多，机器内存干爆，导致系统卡死

为了避免系统过载，你可以通过 环境变量 MAX_JOBS 限制同时运行的编译作业数量。例如：

```
export MAX_JOBS=6
pip install -e .
```

这在你在性能较弱的机器上进行构建时尤其有用。例如，当你使用 WSL 时，它默认只分配总内存的 50%，因此使用 `export MAX_JOBS=1` 可以避免同时编译多个文件并导致内存耗尽。一个副作用是构建过程会变得更慢。

此外，如果你在构建 vLLM 时遇到问题，我们建议使用 NVIDIA PyTorch Docker 镜像。

```
# Use `--ipc=host` to make sure the shared memory is large enough.
docker run --gpus all -it --rm --ipc=host nvcr.io/nvidia/pytorch:23.10-py3
```

如果你不想使用 Docker，建议完整安装 CUDA Toolkit。你可以从官方网站下载并安装它。安装后，设置环境变量 CUDA_HOME 为 CUDA Toolkit 的安装路径，并确保 nvcc 编译器在你的 PATH 中，例如：

```
export CUDA_HOME=/usr/local/cuda
export PATH="${CUDA_HOME}/bin:$PATH"
```

这是一个检查步骤，用于验证 CUDA Toolkit 是否正确安装：

```
nvcc --version # verify that nvcc is in your PATH
${CUDA_HOME}/bin/nvcc --version # verify that nvcc is in your CUDA_HOME
```

不支持的操作系统版本

vLLM 只能在 Linux 上完全运行，但出于开发目的，你仍然可以在其他系统（例如 macOS）上构建它，从而允许导入并提供更便捷的开发环境。二进制文件不会被编译，也无法在非 Linux 系统上运行。

在安装之前，只需禁用 `VLLM_TARGET_DEVICE` 环境变量：

```
export VLLM_TARGET_DEVICE=empty
pip install -e .
```

（2）AMD ROCm

安装前提条件（如果你已经在安装了以下内容的环境/docker 中，可以跳过）：

- ROCm
- PyTorch

**推荐使用AMD Pytorch
Docker镜像简化安装过程**

要安装 PyTorch，你可以从一个新的 docker 镜像开始，例如

`rocm/pytorch:rocm6.3_ubuntu24.04_py3.12_pytorch_release_2.4.0` 或 `rocm/pytorch-nightly`。

如果你使用的是 docker 镜像，可以跳到步骤 3。

另外，你也可以使用 PyTorch wheel 文件安装 PyTorch。你可以查看 PyTorch 安装指南，参考 PyTorch Getting Started。示例：

```
pip uninstall torch -y
pip install --no-cache-dir --pre torch --index-url https://download.pytorch.org/whl/rocm6.3
```

1. 安装 ROCm 的 Triton Flash Attention

按照 ROCm/triton 的说明安装 ROCm 的 Triton Flash Attention（默认使用 triton-mlir 分支）。

```
python3 -m pip install ninja cmake wheel pybind11
pip uninstall -y triton
git clone https://github.com/OpenAI/triton.git
cd triton
git checkout e5be006
cd python
pip3 install .
cd ../../
```

注意 如果在构建 Triton 期间看到与下载软件包相关的 HTTP 错误，请再试一次，因为该 HTTP 错误是间歇性的。

2. 可选地，如果你选择使用 **CK flash attention**，可以为 ROCm 安装 flash attention 按照 ROCm/flash-attention 的说明安装 ROCm 的 flash attention (v2.7.2)。另外，专为 vLLM 使用的 wheel 文件可以在发布版本中找到。

例如，对于 ROCm 6.3，假设你的 gfx 架构是 gfx90a。要获取你的 gfx 架构，运行 `rocminfo | grep gfx`。

```
git clone https://github.com/ROCm/flash-attention.git
cd flash-attention
git checkout b7d29fb
git submodule update --init
GPU_ARCHS="gfx90a" python3 setup.py install
cd ..
```

注意 你可能需要将 "ninja" 版本降级到 1.10，因为在编译 flash-attention-2 时不使用该版本（例如，运行 `pip install ninja==1.10.2.4`）。

构建 vLLM。例如，vLLM 在 ROCm 6.3 上可以通过以下步骤进行构建：

```
pip install --upgrade pip

pip install /opt/rocm/share/amd_smi

pip install --upgrade numba scipy huggingface-hub[cli,hf_transfer] setuptools_scm
pip install "numpy<2"
pip install -r requirements-rocm.txt

export PYTORCH_ROCM_ARCH="gfx90a;gfx942"
python3 setup.py develop
```

先装python依赖，设置环境变量，告诉vllm GPU类型

这可能需要 5 到 10 分钟。目前，`pip install .` 不适用于 ROCm 安装。

提示

默认情况下使用 Triton flash attention。为了进行基准测试，建议在收集性能数据之前执行热身步骤。

目前，Triton flash attention 不支持滑动窗口注意力。如果使用半精度，请使用 CK flash-attention 来支持滑动窗口。

要使用 CK flash-attention 或 PyTorch 原生注意力，请使用以下标志 `export`

`VLLM_USE_TRITON_FLASH_ATTN=0` 关闭 Triton flash attention。

理想情况下，ROCm 版本的 PyTorch 应该与 ROCm 驱动程序版本匹配。

提示

对于 MI300x (gfx942) 用户，为了实现最佳性能，请参考 MI300x 调优指南，获取关于系统和 workflow 级别的性能优化和调优建议。对于 vLLM，请参考 vLLM 性能优化。

(3) Intel XPU

首先，安装 所需的驱动程序和 Intel OneAPI 2024.2 或更高版本。

其次，安装 vLLM XPU 后端构建所需的 Python 包：

```
source /opt/intel/oneapi/setvars.sh
pip install --upgrade pip
pip install -v -r requirements-xpu.txt
```

最后，构建并安装 vLLM XPU 后端：

```
VLLM_TARGET_DEVICE=xpu python setup.py install
```

注意

FP16 是当前 XPU 后端的默认数据类型。BF16 数据类型在 Intel 数据中心 GPU 上受支持，但尚不支持 Intel Arc GPU。

XPU平台目前支持张量并行推理，以及作为Beta功能的管道并行，后者需要配合Ray分布式框架一起使用

3. 使用 Docker 设置

3.1 预构建镜像

(1) NVIDIA CUDA

查看 《使用 vLLM 的官方 Docker 镜像》 以获取使用官方 Docker 镜像的说明。

另一种访问最新代码的方法是使用 Docker 镜像：


```
export VLLM_COMMIT=33f460b17a54acb3b6cc0b03f4a17876cff5eafd # use full commit hash from the main branch
docker pull public.ecr.aws/q9t5s3a7/vllm-ci-postmerge-repo:${VLLM_COMMIT}
```

这些 Docker 镜像仅用于 CI 和测试，不能用于生产环境。它们将在几天后过期。

最新的代码可能包含 bug，并且可能不稳定。请谨慎使用。

(2) AMD ROCm

AMD 为 vLLM 提供的 Infinity 中心提供了一个预构建的、经过优化的 Docker 镜像，旨在验证 AMD Instinct™ MI300X 加速器上的推理性能。

提示

请查看《在 AMD Instinct MI300X 上验证 LLM 推理性能》以获取有关如何使用此预构建 Docker 镜像的说明。

(3) Intel XPU

目前，没有预构建的 XPU 镜像。

3.2 从源代码构建镜像

(1) NVIDIA CUDA

查看《从源代码构建 vLLM 的 Docker 镜像》以获取构建 Docker 镜像的说明。

(2) AMD ROCm

从源代码构建 Docker 镜像是推荐的方式来使用 vLLM 与 ROCm。

(可选) 使用 ROCm 软件栈构建镜像

从 `Dockerfile.rocm_base` 构建一个 Docker 镜像，该镜像设置了 vLLM 所需的 ROCm 软件栈。此步骤是可选的，因为这个 `rocm_base` 镜像通常是预构建的，并且存储在 Docker Hub 上，标签为 `rocm/vllm-dev:base`，以加快用户体验。如果你选择自己构建这个 `rocm_base` 镜像，步骤如下。

重要的是，用户需要使用 BuildKit 启动 Docker 构建。用户可以在调用 `docker build` 命令时将 `DOCKER_BUILDKIT=1` 设置为环境变量，或者用户需要在 Docker 守护进程配置文件 `/etc/docker/daemon.json` 中设置 BuildKit，如下所示，并重新启动守护进程：

```
{
  "features": {
    "buildkit": true
  }
}
```

要在 ROCm 6.3 上为 MI200 和 MI300 系列构建 vLLM，可以使用默认设置：

```
DOCKER_BUILDKIT=1 docker build -f Dockerfile.rocm_base -t rocm/vllm-dev:base .
```

构建 vLLM 镜像

首先，从 `Dockerfile.rocm` 构建一个 Docker 镜像，并从该镜像启动一个 Docker 容器。重要的是，用户需要使用 BuildKit 启动 Docker 构建。用户可以在调用 `docker build` 命令时将 `DOCKER_BUILDKIT=1` 设置为环境变量，或者用户需要在 Docker 守护进程配置文件 `/etc/docker/daemon.json` 中设置 BuildKit，如下所示，并重新启动守护进程：

```
{
  "features": {
    "buildkit": true
  }
}
```

`Dockerfile.rocm` 默认使用 ROCm 6.3，但也支持 ROCm 5.7、6.0、6.1 和 6.2，这些版本存在于较旧的 vLLM 分支中。它提供了灵活性，可以使用以下参数自定义 Docker 镜像的构建：

- **BASE_IMAGE**: 指定在运行 `docker build` 时使用的基础镜像。默认值 `rocm/vllm-dev:base` 是由 AMD 发布和维护的镜像，使用 `Dockerfile.rocm_base` 构建。
- **USE_CYTHON**: 这是一个选项，用于在 Docker 构建时对部分 Python 文件运行 Cython 编译。
- **BUILD_RPD**: 将 RcmProfileData 性能分析工具包含在镜像中。
- **ARG_PYTORCH_ROCM_ARCH**: 允许覆盖基础 Docker 镜像中的 gfx 架构值。

这些值可以通过在运行 `docker build` 时使用 `--build-arg` 选项传递。

要在 ROCm 6.3 上为 MI200 和 MI300 系列构建 vLLM，可以使用默认设置：

```
DOCKER_BUILDKIT=1 docker build -f Dockerfile.rocm -t vllm-rocm .
```

要在 ROCm 6.3 上为 Radeon RX7900 系列 (gfx1100) 构建 vLLM，你应该选择另一个基础镜像：

```
DOCKER_BUILDKIT=1 docker build --build-arg BASE_IMAGE="rocm/vllm-dev:navi_base" -f Dockerfile.rocm -t vllm-rocm .
```

要运行上述 Docker 镜像 `vllm-rocm`，使用以下命令：

```
docker run -it \
--network=host \
--group-add=video \
--ipc=host \
--cap-add=SYS_PTRACE \
--security-opt seccomp=unconfined \
--device /dev/kfd \
--device /dev/dri \
-v <path/to/model>:/app/model \
vllm-rocm \
bash
```

其中，`<path/to/model>` 是模型存储的路径，例如，llama2 或 llama3 模型的权重所在的位置。

** (3) Intel XPU **

```
docker build -f Dockerfile.xpu -t vllm-xpu-env --shm-size=4g .
docker run -it \
```

4. 支持的功能

(1) NVIDIA CUDA

查看《功能 x 硬件兼容性矩阵》以获取功能支持信息。

(2) AMD ROCm

查看《功能 x 硬件兼容性矩阵》以获取功能支持信息。

(3) Intel XPU

XPU 平台支持张量并行推理/服务，并且还支持作为在线服务的 beta 特性——管道并行。我们要求使用 Ray 作为分布式运行时后端。例如，参考执行如下：

```
python -m vllm.entrypoints.openai.api_server \
--model=facebook/opt-13b \
--dtype=bfloat16 \
--device=xpu \
--max_model_len=1024 \
--distributed-executor-backend=ray \
--pipeline-parallel-size=2 \
-tp=8
```

默认情况下，如果系统中未检测到现有的 Ray 实例，将自动启动一个 Ray 实例，`num-gpus` 等于 `parallel_config.world_size`。我们建议在执行之前，正确地启动一个 Ray 集群，可以参考 `examples/online_serving/run_cluster.sh` 辅助脚本。



已关注

👍 0 🗨️ 0 ⭐ 0 💬 0 📄 分享 ...

专栏目录

已订阅

是一个快速且易于使用的库，用于 LLM 推理和服务，可以和HuggingFace 无缝集成。vLLM利用了全新的注意力算法「PagedAttention」， 有效地管理注意力键和值。在吞吐量方面， vLLM的性能比HuggingFace Transformers(H...)高出 24 倍，文本生成推理（TGI）高出3.5倍。是伯克利大学LMSYS组织开源的大语言模型高速推理框架，旨在极大地提升实时场景下的语言模型服务的吞吐与内存使用效率。

vLLM ([https://github.com/ vllm-project/ vllm](https://github.com/vllm-project/vllm)) 是一个快速且易于使用的库，用于进行大型语言模型的推理和部署。

本地部署 vllm